
Data Structures

BS (CS) _Fall_2025

Lab_07 Task



Learning Objectives:

1. Sorting using Arrays and Linked Lists

Task 1:

Problem Statement:

Write a C++ program to implement Merge Sort on a singly linked list. The program should:

Requirements:

Define a Node class with data and next members.

Implement the following functions:

- **Node* mergeSort(Node* head)** → Recursively sorts the list using Merge Sort.
- **Node* split(Node* head)** → Splits the list into two halves using the slow and fast pointer approach.
- **Node* merge(Node* first, Node* second)** → Merges two sorted linked lists into one.
- **void printList(Node* head)** → Displays the linked list in proper format.

Complexity Analysis:

- Write a function that displays the size of list and show the number of iterations it takes for your Merge Sort algorithm to sort a given list.
- Perform Complexity Analysis on your code. Show your work on a piece of paper.

Algorithm

MergeSort Function

- Check if the linked list is empty or has only one node. If yes, return it (already sorted).
- Otherwise, divide the list into two halves using the Split function.
- Recursively apply MergeSort on the first half.
- Recursively apply MergeSort on the second half.
- Combine the two sorted halves using the Merge function and return the result.

Split Function

- Use two pointers (slow and fast) starting from the head of the list.
- Move the fast pointer two steps at a time and the slow pointer one step at a time until fast reaches the end.
- The slow pointer will then point to the middle node.
- Break the linked list into two halves at the middle.
- Return the starting node of the second half.

Merge Function

- If one of the lists is empty, return the other list.
- Otherwise, compare the current nodes of both lists.
- Select the node with the smaller value and link it to the result.

- Recursively continue merging the remaining nodes from both lists.
- Return the merged sorted list

Expected Output (Example):

Original List: 9 -> 8 -> 5 -> 2

Sorted List: 2 -> 5 -> 8 -> 9

Test Cases:

```
// Test empty list
TEST(MergeSortTest, EmptyListInt) {
    LinkedList<int> list;
    list.mergeSort();
    // Empty list should remain empty after sorting
    EXPECT_TRUE(list.isEmpty());
}

// Test single element list
TEST(MergeSortTest, SingleElementInt) {
    LinkedList<int> list;
    list.insertAtHead(42);
    list.mergeSort();

    int expected[] = {42};
    Node<int>* current = list.getHead();
    for (int i = 0; i < 1; i++) {
        EXPECT_NE(current, nullptr);
        EXPECT_EQ(current->data, expected[i]);
        current = current->next;
    }
    EXPECT_EQ(current, nullptr);
}

// Test two elements - already sorted
TEST(MergeSortTest, TwoElementsSortedInt) {
    LinkedList<int> list;
    list.insertAtHead(2);
    list.insertAtHead(1);
    list.mergeSort();

    int expected[] = {1, 2};
    Node<int>* current = list.getHead();
```

```

    for (int i = 0; i < 2; i++) {
        EXPECT_NE(current, nullptr);
        EXPECT_EQ(current->data, expected[i]);
        current = current->next;
    }
    EXPECT_EQ(current, nullptr);
}

// Test two elements - reverse order
TEST(MergeSortTest, TwoElementsReverseInt) {
    LinkedList<int> list;
    list.insertAtHead(3);
    list.insertAtHead(5);
    list.mergeSort();

    int expected[] = {3, 5};
    Node<int>* current = list.getHead();
    for (int i = 0; i < 2; i++) {
        EXPECT_NE(current, nullptr);
        EXPECT_EQ(current->data, expected[i]);
        current = current->next;
    }
    EXPECT_EQ(current, nullptr);
}

// Test odd number of elements
TEST(MergeSortTest, OddElementsInt) {
    LinkedList<int> list;
    int input[] = {64, 34, 25, 12, 22};
    for (int i = 0; i < 5; i++) {
        list.insertAtHead(input[i]);
    }
    list.mergeSort();

    int expected[] = {12, 22, 25, 34, 64};
    Node<int>* current = list.getHead();
    for (int i = 0; i < 5; i++) {
        EXPECT_NE(current, nullptr);
        EXPECT_EQ(current->data, expected[i]);
        current = current->next;
    }
    EXPECT_EQ(current, nullptr);
}

// Test even number of elements

```

```

TEST(MergeSortTest, EvenElementsInt) {
    LinkedList<int> list;
    int input[] = {38, 27, 43, 3, 9, 82};
    for (int i = 0; i < 6; i++) {
        list.insertAtHead(input[i]);
    }
    list.mergeSort();

    int expected[] = {3, 9, 27, 38, 43, 82};
    Node<int>* current = list.getHead();
    for (int i = 0; i < 6; i++) {
        EXPECT_NE(current, nullptr);
        EXPECT_EQ(current->data, expected[i]);
        current = current->next;
    }
    EXPECT_EQ(current, nullptr);
}

// Test already sorted list
TEST(MergeSortTest, AlreadySortedInt) {
    LinkedList<int> list;
    int input[] = {7, 6, 5, 4, 3, 2, 1}; // Insert in reverse to get sorted order
    for (int i = 0; i < 7; i++) {
        list.insertAtHead(input[i]);
    }
    list.mergeSort();

    int expected[] = {1, 2, 3, 4, 5, 6, 7};
    Node<int>* current = list.getHead();
    for (int i = 0; i < 7; i++) {
        EXPECT_NE(current, nullptr);
        EXPECT_EQ(current->data, expected[i]);
        current = current->next;
    }
    EXPECT_EQ(current, nullptr);
}

// Test reverse sorted list
TEST(MergeSortTest, ReverseSortedInt) {
    LinkedList<int> list;
    int input[] = {5, 6, 7, 8, 9, 10}; // Insert to get reverse order
    for (int i = 0; i < 6; i++) {
        list.insertAtHead(input[i]);
    }
    list.mergeSort();
}

```

```

    int expected[] = {5, 6, 7, 8, 9, 10};
    Node<int>* current = list.getHead();
    for (int i = 0; i < 6; i++) {
        EXPECT_NE(current, nullptr);
        EXPECT_EQ(current->data, expected[i]);
        current = current->next;
    }
    EXPECT_EQ(current, nullptr);
}

// Test with duplicate elements
TEST(MergeSortTest, DuplicateElementsInt) {
    LinkedList<int> list;
    int input[] = {5, 1, 9, 2, 8, 2, 5, 5};
    for (int i = 0; i < 8; i++) {
        list.insertAtHead(input[i]);
    }
    list.mergeSort();

    int expected[] = {1, 2, 2, 5, 5, 5, 8, 9};
    Node<int>* current = list.getHead();
    for (int i = 0; i < 8; i++) {
        EXPECT_NE(current, nullptr);
        EXPECT_EQ(current->data, expected[i]);
        current = current->next;
    }
    EXPECT_EQ(current, nullptr);
}

// Test with negative numbers
TEST(MergeSortTest, NegativeNumbersInt) {
    LinkedList<int> list;
    int input[] = {2, 0, -1, 7, -5, -3};
    for (int i = 0; i < 6; i++) {
        list.insertAtHead(input[i]);
    }
    list.mergeSort();

    int expected[] = {-5, -3, -1, 0, 2, 7};
    Node<int>* current = list.getHead();
    for (int i = 0; i < 6; i++) {
        EXPECT_NE(current, nullptr);
        EXPECT_EQ(current->data, expected[i]);
        current = current->next;
    }

```

```

    }
    EXPECT_EQ(current, nullptr);
}

// Test large list
TEST(MergeSortTest, LargeListInt) {
    LinkedList<int> list;
    int input[] = {125, 300, 25, 50, 75, 150, 2, 3, 1, 4, 200, 100};
    for (int i = 0; i < 12; i++) {
        list.insertAtHead(input[i]);
    }
    list.mergeSort();

    int expected[] = {1, 2, 3, 4, 25, 50, 75, 100, 125, 150, 200, 300};
    Node<int>* current = list.getHead();
    for (int i = 0; i < 12; i++) {
        EXPECT_NE(current, nullptr);
        EXPECT_EQ(current->data, expected[i]);
        current = current->next;
    }
    EXPECT_EQ(current, nullptr);
}

// Test using insertAtTail
TEST(MergeSortTest, InsertAtTailInt) {
    LinkedList<int> list;
    int input[] = {64, 34, 25, 12, 22};
    for (int i = 0; i < 5; i++) {
        list.insertAtTail(input[i]);
    }
    list.mergeSort();

    int expected[] = {12, 22, 25, 34, 64};
    Node<int>* current = list.getHead();
    for (int i = 0; i < 5; i++) {
        EXPECT_NE(current, nullptr);
        EXPECT_EQ(current->data, expected[i]);
        current = current->next;
    }
    EXPECT_EQ(current, nullptr);
}

// Float data type tests
TEST(MergeSortTest, FloatElements) {
    LinkedList<float> list;

```

```

float input[] = {0.9f, 2.1f, 4.8f, 1.2f, 3.5f};
for (int i = 0; i < 5; i++) {
    list.insertAtHead(input[i]);
}
list.mergeSort();

float expected[] = {0.9f, 1.2f, 2.1f, 3.5f, 4.8f};
Node<float>* current = list.getHead();
for (int i = 0; i < 5; i++) {
    EXPECT_NE(current, nullptr);
    EXPECT_FLOAT_EQ(current->data, expected[i]);
    current = current->next;
}
EXPECT_EQ(current, nullptr);
}

// Float with negative values
TEST(MergeSortTest, FloatNegativeElements) {
    LinkedList<float> list;
    float input[] = {4.2f, 0.0f, -1.1f, 3.7f, -2.5f};
    for (int i = 0; i < 5; i++) {
        list.insertAtHead(input[i]);
    }
    list.mergeSort();

    float expected[] = {-2.5f, -1.1f, 0.0f, 3.7f, 4.2f};
    Node<float>* current = list.getHead();
    for (int i = 0; i < 5; i++) {
        EXPECT_NE(current, nullptr);
        EXPECT_FLOAT_EQ(current->data, expected[i]);
        current = current->next;
    }
    EXPECT_EQ(current, nullptr);
}

// Double data type tests
TEST(MergeSortTest, DoubleElements) {
    LinkedList<double> list;
    double input[] = {2.718, 3.141, 5.555, 1.234, 9.876};
    for (int i = 0; i < 5; i++) {
        list.insertAtHead(input[i]);
    }
    list.mergeSort();

    double expected[] = {1.234, 2.718, 3.141, 5.555, 9.876};

```



```

Node<double>* current = list.getHead();
for (int i = 0; i < 5; i++) {
    EXPECT_NE(current, nullptr);
    EXPECT_DOUBLE_EQ(current->data, expected[i]);
    current = current->next;
}
EXPECT_EQ(current, nullptr);
}

// Double with scientific notation
TEST(MergeSortTest, DoubleScientificNotation) {
    LinkedList<double> list;
    double input[] = {1.1e-1, 8.9e0, 4.7e1, 2.3e-2, 1.5e3};
    for (int i = 0; i < 5; i++) {
        list.insertAtHead(input[i]);
    }
    list.mergeSort();

    double expected[] = {2.3e-2, 1.1e-1, 8.9e0, 4.7e1, 1.5e3};
    Node<double>* current = list.getHead();
    for (int i = 0; i < 5; i++) {
        EXPECT_NE(current, nullptr);
        EXPECT_DOUBLE_EQ(current->data, expected[i]);
        current = current->next;
    }
    EXPECT_EQ(current, nullptr);
}

// Test single element with different data types
TEST(MergeSortTest, SingleElementFloat) {
    LinkedList<float> list;
    list.insertAtHead(42.5f);
    list.mergeSort();

    float expected[] = {42.5f};
    Node<float>* current = list.getHead();
    for (int i = 0; i < 1; i++) {
        EXPECT_NE(current, nullptr);
        EXPECT_FLOAT_EQ(current->data, expected[i]);
        current = current->next;
    }
    EXPECT_EQ(current, nullptr);
}

TEST(MergeSortTest, SingleElementDouble) {

```

```

LinkedList<double> list;
list.insertAtHead(42.123456789);
list.mergeSort();

double expected[] = {42.123456789};
Node<double>* current = list.getHead();
for (int i = 0; i < 1; i++) {
    EXPECT_NE(current, nullptr);
    EXPECT_DOUBLE_EQ(current->data, expected[i]);
    current = current->next;
}
EXPECT_EQ(current, nullptr);
}

// Test empty lists for different data types
TEST(MergeSortTest, EmptyListFloat) {
    LinkedList<float> list;
    list.mergeSort();
    EXPECT_TRUE(list.isEmpty());
}

TEST(MergeSortTest, EmptyListDouble) {
    LinkedList<double> list;
    list.mergeSort();
    EXPECT_TRUE(list.isEmpty());
}

```